Why and How Trello Moved from RabbitMQ to Kafka

Source: Arpit Bhayani Dissection • Trello Infrastructure
Focus: WebSocket Scaling, Split-Brain Catastrophes & Message Queues

1. The Problem Space: Real-Time Board Synchronization

At Trello, instantaneous multi-user collaboration is the core product promise. When a user creates a card, drags a task across a Kanban column, or assigns a teammate, every connected client viewing that board must receive the event in real time via WebSockets without refreshing.

To deliver updates across millions of concurrent users, Trello originally migrated from Redis Pub/Sub to a large **15-node RabbitMQ cluster** divided into inbound and outbound tiers using the Shovel plugin.

2. The Three Architectural Bottlenecks of RabbitMQ

```
+-------------------------------------------------------------------------------------------------+
|                                LEGACY RABBITMQ CLUSTER PAIN POINTS                              |
+-------------------------------------------------------------------------------------------------+
  1. Split-Brain Failures:
     - Erlang Mnesia network partitions caused cluster split-brains.
     - Required full manual cluster restarts, losing all in-flight real-time events.

  2. Ephemeral Queue Churn:
     - Thousands of WebSocket clients constantly connecting/disconnecting.
     - RabbitMQ had to create and destroy thousands of short-lived Erlang queue processes per minute.
     - Caused severe Erlang garbage collection freezes and memory fragmentation.

  3. Fan-out Binding Duplication:
     - High collaborator counts forced the exchange to clone messages across 50+ queue buffers in RAM.
```

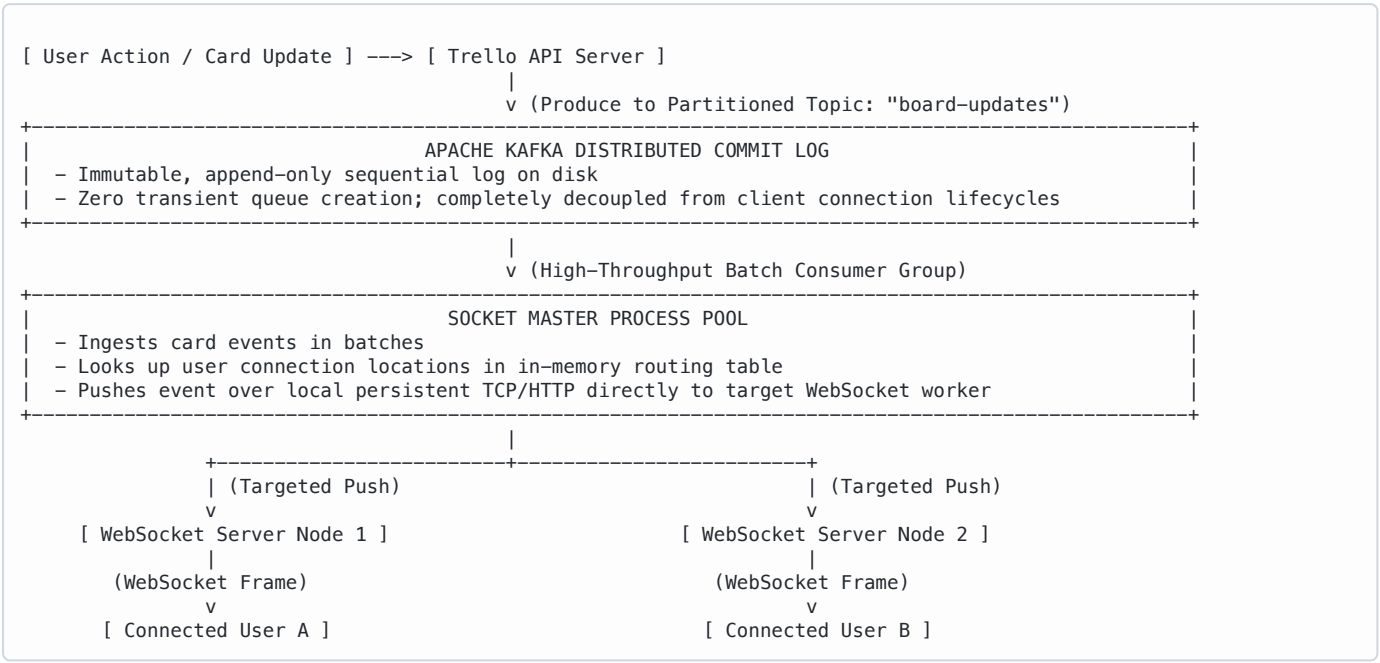
> **The Fundamental Structural Mismatch:** Traditional message brokers (RabbitMQ) track state per message and per queue. Forcing an AMQP broker to maintain millions of ephemeral client queues creates an unmanageable state explosion.

The Kafka Architecture & The Socket Master Topology

Source: Arpit Bhayani Dissection • Trello Infrastructure
Focus: Append-Only Logs, Socket Masters & Connection Decoupling

3. The Kafka Architecture: The Socket Master Topology

Trello replaced the complex RabbitMQ queue mesh with a clean, decoupled architecture powered by **Apache Kafka**:



4. Key Structural Advantages of Kafka's Log Model

- **Zero Queue Creation:** Kafka topics are partitioned statically. Client connections can fluctuate wildly without creating a single new entity inside the broker.
- **Consumer-Side Offset Tracking:** Kafka brokers do not track which clients received which messages. The Socket Master manages its own read offsets, keeping Kafka ultra-fast and stateless.
- **Zero-Copy OS Page Cache:** Messages are streamed from disk to network sockets using the Linux `sendfile` system call, bypassing JVM user space memory completely.

Production Impact & First-Principles Lessons

5. Production Impact: 5x Cost Reduction & Zero Outages

Deploying the Kafka + Socket Master architecture revolutionized Trello's real-time messaging layer:

Architecture Dimension	Legacy RabbitMQ Mesh	Kafka + Socket Master	Empirical Gain
Infrastructure Cost	15-node heavy broker cluster	Lean Kafka cluster + Socket Masters	**5x direct cost reduction (~80% savings)**
CPU Utilization Efficiency	High CPU churn from Erlang GC and queue lifecycle	High-throughput batch sequential I/O	**2x improvement in CPU efficiency**
Memory Usage	High; message duplication across transient queues	Zero-copy kernel page cache buffers	**33% reduction in memory footprint**
Outage Frequency	Recurring split-brain manual cluster resets	Zero split-brain cluster crashes	**Near-zero major real-time outages**

6. First-Principles System Design Takeaways

Design Dimension	Message Queue (RabbitMQ)	Distributed Log (Kafka)
Mental Model	Smart broker, dumb consumer (broker tracks state)	Dumb broker, smart consumer (append-only log)
Ephemeral Clients	Anti-pattern; transient queues exhaust broker state	Natural fit; consumers decouple connection lifecycle from storage
Fan-Out Scaling	Copies messages into N distinct memory queues	Single log read by multiple independent consumer groups
Resilience	Susceptible to split-brain network partitions	Strict quorum consensus (KRaft/Raft), partition tolerant